

Disjoint Set Union (DSU) / Union-Find

1. Overview

Disjoint Set Union (DSU), also called **Union-Find**, is a data structure that tracks a set of elements partitioned into **disjoint (non-overlapping) subsets**. It supports two primary operations efficiently:

- **Find**: Determine which subset a particular element belongs to (returns the representative/root of the set).
- **Union**: Merge two subsets into a single subset.

DSU is the backbone of many graph algorithms, especially those dealing with connectivity, cycle detection, and minimum spanning trees. It is simple, fast, and nearly **$O(1)$ amortized** per operation with optimizations.

The key idea: each set is represented as a **tree**, where the root of the tree is the "representative" of the set. All elements in the same set point (directly or indirectly) to the same root.

2. Intuition

Simple Explanation

Imagine a group of people at a party. Initially, everyone is a stranger to everyone else — each person is their own "group." When two people shake hands and become friends, their groups merge. If later you want to know if two people are in the same friend group, you just trace back to the group leader.

DSU does exactly this:

- Each element starts as its own parent (leader of its own group).
- When two elements are connected, we make one group's leader point to the other's leader (or vice versa).
- To check if two elements are connected, we find who their respective leaders are. If the leaders are the same, they are connected.

Analogy: The Family Tree

Think of a DSU as a collection of **family trees**:

- Every person has a **parent pointer** (initially pointing to themselves).
- The **root** of the tree is the eldest ancestor (the representative).
- **Find** is like asking "who is the eldest ancestor of this person?" — you climb the parent pointers until you reach someone who is their own parent.
- **Union** is like merging two families: you make the eldest ancestor of one family point to the eldest ancestor of the other.

Step-by-Step Reasoning

1. **Initialization**: Every element is its own set. $\text{parent}[i] = i$ for all i .

2. **Find(x)**: Follow `parent[x] → parent[parent[x]] → ...` until you reach an element `p` where `parent[p] == p`. That `p` is the root/representative.
3. **Union(x, y)**: Find roots of `x` and `y`. If they are different, make one root point to the other.
4. **Optimization (Path Compression)**: During Find, make every node on the path point directly to the root. This flattens the tree.
5. **Optimization (Union by Size/Rank)**: Always attach the smaller tree under the larger tree. This keeps trees shallow.

Why It Works

- The **parent pointer** structure guarantees that each set has exactly one root (representative).
- **Path compression** ensures that nearly all nodes point directly to the root, making future Finds $O(1)$.
- **Union by size/rank** ensures that tree height is at most $O(\log n)$, and with path compression, the amortized complexity is $O(\alpha(n))$ — the inverse Ackermann function, which is essentially constant for all practical inputs.

3. When to Use It

Use DSU when you need to:

Situation	Description
Dynamic connectivity	Connect components and query whether two elements are in the same set
Cycle detection (undirected)	As you add edges, check if the two endpoints already belong to the same set
Number of components	Track how many disjoint sets exist after a series of union operations
Minimum Spanning Tree	Kruskal's algorithm uses DSU to add edges without forming cycles
Equivalence relations	Problems where "if A is related to B and B to C, then A is related to C" (transitive closure)
Grid connectivity	Merging adjacent cells in a grid (2D to 1D mapping)
Offline queries with rollback	When you need to undo unions (DSU with rollback)

Common Trigger Phrases from Problems

- "Connected components"
- "Merge groups"
- "Find if two elements are in the same set"
- "Number of connected components after operations"
- "Detect if adding an edge creates a cycle"
- "Minimum cost to connect all nodes"
- "Accounts/people with common information"
- "Redundant connection"
- "Number of islands" (when using DSU variant)

- "Dynamic connectivity"
- "Graph is connected or not"
- "Grouping based on some relation"

4. When Not to Use It

Situation	Why Not	Alternative
Dynamic connectivity with deletions	DSU does not support deleting edges (without rollback/offline tricks)	Link-Cut Tree, Euler Tour Tree
Directed graph connectivity	DSU only works for undirected connectivity	Kosaraju, Tarjan (SCC)
Path queries between nodes	DSU tells you if two nodes are connected, not the path	BFS, DFS, Union-Find with path queries (offline)
Distance between nodes	DSU does not store distances	Weighted Union-Find, BFS, Dijkstra
Frequent queries without updates	Simpler preprocessing (DFS connectivity) may be faster	Precompute connected components via DFS
Small, static graphs	DSU overhead is unnecessary	Simple BFS/DFS from each query
When you need the actual components	DSU gives representatives, but to list all elements in a set you need extra work	DFS to collect components

Common Wrong Assumptions

- ❌ "DSU works for directed graphs too" — DSU models undirected connectivity. For directed graphs, use SCC algorithms.
- ❌ "DSU can handle edge deletions" — Standard DSU is insert-only. You need special techniques (rollback, divide-and-conquer) for deletions.
- ❌ "DSU can give me the path between two nodes" — DSU only tells you if they are connected, not the path.
- ❌ "DSU works for weighted edges and distances" — DSU does not track edge weights by default. Use Weighted DSU or Kruskal's algorithm.

5. Core Concepts

5.1 Parent Array

What it is: An array `parent[]` where `parent[i]` stores the parent of element `i`. If `parent[i] == i`, then `i` is the root (representative) of its set.

Why it matters: The parent array is the core data structure of DSU. All operations are defined in terms of following parent pointers.

Example:

```
Initial: parent = [0, 1, 2, 3, 4]
After union(0, 1): parent = [0, 0, 2, 3, 4] (1's parent is 0)
After union(2, 3): parent = [0, 0, 2, 2, 4] (3's parent is 2)
After union(1, 3): parent = [0, 0, 0, 2, 4] (2's parent is 0 after union of roots)
```

5.2 Find Operation

What it is: `find(x)` returns the root (representative) of the set containing `x`. It climbs the parent pointers until it reaches a node whose parent is itself.

Why it matters: Find is the fundamental query operation. It tells us which set an element belongs to and is used by both Union and connectivity checks.

Implementation idea:

```
int find(int x) {
    while (parent[x] != x)
        x = parent[x];
    return x;
}
```

5.3 Union Operation

What it is: `union(x, y)` merges the sets containing `x` and `y` into one. It finds the roots of both and makes one root point to the other.

Why it matters: Union is the fundamental update operation. It allows us to dynamically connect components.

Implementation idea:

```
void unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx != ry)
        parent[ry] = rx; // or vice versa
}
```

5.4 Path Compression

What it is: During `find(x)`, after finding the root, we set the parent of every node visited along the path directly to the root. This flattens the tree.

Why it matters: Path compression drastically reduces the height of trees. Future Finds on any node in the path become $O(1)$. This is the key optimization that makes DSU nearly constant time.

Example:

```
Before find(4): parent = [0, 0, 0, 2, 3]
find(4) climbs: 4 → 3 → 2 → 0
After find(4): parent = [0, 0, 0, 0, 0] (4 and 3 now point directly to 0)
```

5.5 Union by Size / Union by Rank

What it is: When merging two sets, always attach the **smaller** tree (by size or rank/height) under the **larger** tree.

- **Union by Size:** Track `size[i]` = number of elements in the set rooted at `i`. Attach smaller size to larger size.
- **Union by Rank:** Track `rank[i]` = approximate height of the tree rooted at `i`. Attach lower rank to higher rank.

Why it matters: Without this optimization, Union can create tall, skinny trees ($O(n)$ height). With union by size/rank, tree height is at most $O(\log n)$. Combined with path compression, the amortized complexity becomes $O(\alpha(n))$.

Example:

```
Set A: size 5, root = 0
Set B: size 2, root = 3
Union by size: parent[3] = 0 (attach smaller to larger)
Now size[0] = 7
```

5.6 Cycle Detection

What it is: While adding edges to a graph, for each edge (u, v) , if `find(u) == find(v)`, then adding this edge creates a cycle (since u and v are already connected).

Why it matters: This is the key insight used in Kruskal's MST algorithm and the Redundant Connection problem.

5.7 Number of Components

What it is: Track how many disjoint sets exist. Initially, if there are `n` elements, there are `n` components. Each time a successful union merges two different sets, the component count decreases by 1.

Why it matters: Many problems ask for the number of connected components after a series of operations.

5.8 DSU on Grid (2D to 1D Mapping)

What it is: When working with a grid, we map each cell (r, c) to a unique ID `r * cols + c`. Then we apply DSU operations on these IDs.

Why it matters: Grid problems (number of islands, regions, etc.) can be solved with DSU by unioning adjacent cells that satisfy some condition.

5.9 DSU Rollback (Undo)

What it is: A version of DSU that supports undoing the last union operation. This requires storing the state changes on a stack and not using path compression (since it's destructive).

Why it matters: Used in offline divide-and-conquer algorithms, dynamic connectivity problems with time windows, and problems where you need to explore/add/remove connections.

5.10 DSU on Tree (Tree DP with DSU)

What it is: Processing tree nodes in a specific order (often from leaves to root or reverse) and using DSU to merge children's information into the parent. Sometimes called "DSU on tree" or "Sack" (DSU on tree — a different technique, also known as "small to large merging").

Why it matters: This is a powerful technique for answering queries on trees offline, especially subtree queries. The "small to large" merging principle ensures $O(n \log n)$ overall complexity.

6. Step-by-Step Algorithm

Basic DSU (without optimizations)

1. **Initialize:** Create an array `parent` of size `n`. Set `parent[i] = i` for all `i` from `0` to `n-1`.
2. **Find(x):**
 - If `parent[x] == x`, return `x` (it's a root).
 - Otherwise, recursively call `find(parent[x])` and return its result.
3. **Union(x, y):**
 - Let `rootX = find(x)` and `rootY = find(y)`.
 - If `rootX == rootY`, do nothing (they are already in the same set).
 - Otherwise, set `parent[rootY] = rootX` (or vice versa).

DSU with Path Compression (optimized Find)

1. **Initialize:** Same as basic.
2. **Find(x):**
 - If `parent[x] != x`, set `parent[x] = find(parent[x])` (path compression).
 - Return `parent[x]`.
3. **Union(x, y):** Same as basic, but with optimized Find.

DSU with Union by Size (optimized Union)

1. **Initialize:** Create `parent` array of size `n` and `size` array of size `n` (filled with 1).
 - Set `parent[i] = i` for all `i`.
2. **Find(x)** with path compression (same as above).
3. **Union(x, y):**
 - Let `rootX = find(x)` and `rootY = find(y)`.
 - If `rootX == rootY`, return (already connected).

- If `size[rootX] < size[rootY]`, swap `rootX` and `rootY` (ensuring `rootX` is the larger set).
- Set `parent[rootY] = rootX`.
- Update `size[rootX] += size[rootY]`.

DSU with Rollback (for undo operations)

1. **Initialize:** Create `parent` and `size` arrays. Do NOT use path compression.
2. **Find(x)** without path compression: climb parent pointers iteratively.
3. **Union(x, y)** with rollback support:
 - Find roots without path compression.
 - If roots are same, push a "no-op" marker to the stack and return false.
 - If `size[rootX] < size[rootY]`, swap.
 - Push the old state (`rootY, parent[rootY], rootX, size[rootX]`) to the stack.
 - Set `parent[rootY] = rootX`.
 - Update `size[rootX] += size[rootY]`.
 - Return true.
4. **Rollback():** Pop the top state from the stack and restore the old values.

7. Dry Run

Problem: Given 6 elements (0-5), perform the following unions and check connectivity.

Operations:

1. union(0, 1)
2. union(2, 3)
3. union(1, 2)
4. union(4, 5)
5. query: connected(0, 3)?
6. query: connected(0, 4)?

Initial State

Index	0	1	2	3	4	5
parent	0	1	2	3	4	5
size	1	1	1	1	1	1

Step 1: union(0, 1)

- `find(0) = 0, find(1) = 1`
- `size[0] = 1, size[1] = 1` → tie, attach 1 under 0
- `parent[1] = 0, size[0] = 2`

Index	0	1	2	3	4	5
parent	0	0	2	3	4	5
size	2	1	1	1	1	1

Step 2: union(2, 3)

- $\text{find}(2) = 2, \text{find}(3) = 3$
- $\text{size}[2] = 1, \text{size}[3] = 1 \rightarrow$ attach 3 under 2
- $\text{parent}[3] = 2, \text{size}[2] = 2$

Index	0	1	2	3	4	5
parent	0	0	2	2	4	5
size	2	1	2	1	1	1

Step 3: union(1, 2)

- $\text{find}(1): 1 \rightarrow \text{parent}[1] = 0 \rightarrow \text{find}(0) = 0 \rightarrow$ return 0 (with path compression: $\text{parent}[1] = 0$ already)
- $\text{find}(2) = 2$
- $\text{rootX} = 0$ (size = 2), $\text{rootY} = 2$ (size = 2) $\rightarrow$ tie, attach 2 under 0
- $\text{parent}[2] = 0, \text{size}[0] = 4$

Index	0	1	2	3	4	5
parent	0	0	0	2	4	5
size	4	1	2	1	1	1

Step 4: union(4, 5)

- $\text{find}(4) = 4, \text{find}(5) = 5$
- $\text{size}[4] = 1, \text{size}[5] = 1 \rightarrow$ attach 5 under 4
- $\text{parent}[5] = 4, \text{size}[4] = 2$

Index	0	1	2	3	4	5
parent	0	0	0	2	4	4
size	4	1	2	1	2	1

Step 5: connected(0, 3)?

- $\text{find}(0) = 0$
- $\text{find}(3): 3 \rightarrow \text{parent}[3] = 2 \rightarrow \text{find}(2) = 0$ (path compression: $\text{parent}[3] = 0$)
- Root of 0 = 0, Root of 3 = 0 $\rightarrow$ **Yes, they are connected**

Step 6: connected(0, 4)?

- $\text{find}(0) = 0$
- $\text{find}(4) = 4$
- Root of 0 = 0, Root of 4 = 4 $\rightarrow$ **No, they are not connected**

Final State (with path compression applied during finds)

Index	0	1	2	3	4	5
parent	0	0	0	0	4	4
size	4	1	2	1	2	1

Components: {0, 1, 2, 3} and {4, 5} → 2 components.

8. C++ Implementation

Basic DSU Class (with Path Compression and Union by Size)

```
#include <bits/stdc++.h>
using namespace std;

class DSU {
private:
    vector<int> parent, sz; // sz = size of each set (by number of elements)

public:
    DSU(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        for (int i = 0; i < n; i++)
            parent[i] = i;
    }

    // Find with path compression
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]); // path compression
        return parent[x];
    }

    // Union by size
    bool unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false; // already connected

        // Attach smaller tree under larger tree
        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
        return true;
    }

    // Check if two elements are in the same set
    bool same(int x, int y) {
        return find(x) == find(y);
    }
}
```

```

// Get the size of the set containing x
int size(int x) {
    return sz[find(x)];
}

// Get number of disjoint sets
int count() {
    int cnt = 0;
    for (int i = 0; i < (int)parent.size(); i++)
        if (parent[i] == i) cnt++;
    return cnt;
}
};

// ===== Example Usage =====
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 7;
    DSU dsu(n);

    // Perform unions
    dsu.unite(0, 1);
    dsu.unite(1, 2);
    dsu.unite(3, 4);
    dsu.unite(5, 6);

    // Check connectivity
    cout << "0 and 2 connected? " << (dsu.same(0, 2) ? "Yes" : "No") << "\n";
// Yes
    cout << "0 and 3 connected? " << (dsu.same(0, 3) ? "Yes" : "No") << "\n";
// No
    cout << "Number of components: " << dsu.count() << "\n";
// 3

    // Union more
    dsu.unite(2, 3);
    cout << "After union(2,3):\n";
    cout << "0 and 3 connected? " << (dsu.same(0, 3) ? "Yes" : "No") << "\n";
// Yes
    cout << "Number of components: " << dsu.count() << "\n";
// 2

    // Size of set containing 0
    cout << "Size of set {0,1,2,3,4}: " << dsu.size(0) << "\n";
// 5

    return 0;
}

```

DSU with Union by Rank

```
#include <bits/stdc++.h>
using namespace std;

class DSU {
private:
    vector<int> parent, rank;

public:
    DSU(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        for (int i = 0; i < n; i++)
            parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    bool unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;

        // Union by rank: attach lower rank tree under higher rank tree
        if (rank[rx] < rank[ry])
            parent[rx] = ry;
        else if (rank[rx] > rank[ry])
            parent[ry] = rx;
        else {
            parent[ry] = rx;
            rank[rx]++; // rank increases only when equal
        }
        return true;
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    int count() {
        int cnt = 0;
        for (int i = 0; i < (int)parent.size(); i++)
            if (parent[i] == i) cnt++;
        return cnt;
    }
};
```

DSU with Rollback (Undo Support)

```

#include <bits/stdc++.h>
using namespace std;

struct RollbackDSU {
    vector<int> parent, sz;
    vector<tuple<int, int, int, int>> history; // {child, old_parent, root,
old_size}

    RollbackDSU(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    // Find WITHOUT path compression (iterative)
    int find(int x) const {
        while (parent[x] != x) x = parent[x];
        return x;
    }

    // Union with rollback support
    bool unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) {
            history.emplace_back(-1, -1, -1, -1); // marker for no-op
            return false;
        }

        // Union by size: attach smaller under larger
        if (sz[rx] < sz[ry]) swap(rx, ry);

        // Save state: (child, child's old parent, root, root's old size)
        history.emplace_back(ry, parent[ry], rx, sz[rx]);

        parent[ry] = rx;
        sz[rx] += sz[ry];
        return true;
    }

    // Rollback the last union
    void rollback() {
        auto [child, oldParent, root, oldSize] = history.back();
        history.pop_back();
        if (child == -1) return; // no-op, nothing to restore
        parent[child] = oldParent;
        sz[root] = oldSize;
    }

    // Get snapshot size (number of unions performed)
    int snapshot() const {
        return (int)history.size();
    }
}

```

```

// Rollback to a specific snapshot
void rollbackTo(int snap) {
    while ((int)history.size() > snap)
        rollback();
}

bool same(int x, int y) {
    return find(x) == find(y);
}
};

```

DSU on Grid (2D to 1D Mapping)

```

#include <bits/stdc++.h>
using namespace std;

class GridDSU {
private:
    int rows, cols;
    vector<int> parent, sz;
    int components;

    int id(int r, int c) const {
        return r * cols + c;
    }

public:
    GridDSU(int r, int c) : rows(r), cols(c) {
        int n = r * c;
        parent.resize(n);
        sz.resize(n, 1);
        components = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    bool unite(int r1, int c1, int r2, int c2) {
        int x = id(r1, c1), y = id(r2, c2);
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;

        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
        components--;
    }
};

```

```

        return true;
    }

    bool same(int r1, int c1, int r2, int c2) {
        return find(id(r1, c1)) == find(id(r2, c2));
    }

    int getComponents() const {
        return components;
    }

    int getSize(int r, int c) {
        return sz[find(id(r, c))];
    }
};

```

9. Python Implementation

Basic DSU Class

```

class DSU:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x: int) -> int:
        """Find with path compression (iterative)"""
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]] # path halving
            x = self.parent[x]
        return x

    # Alternative: recursive find
    # def find(self, x: int) -> int:
    #     if self.parent[x] != x:
    #         self.parent[x] = self.find(self.parent[x])
    #     return self.parent[x]

    def unite(self, x: int, y: int) -> bool:
        """Union by size. Returns True if merged, False if already connected."""
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False

        # Attach smaller tree under larger tree
        if self.size[rx] < self.size[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        self.size[rx] += self.size[ry]
        return True

```

```

def same(self, x: int, y: int) -> bool:
    return self.find(x) == self.find(y)

def component_size(self, x: int) -> int:
    return self.size[self.find(x)]

def count(self) -> int:
    """Return number of disjoint sets"""
    return sum(1 for i in range(len(self.parent)) if self.parent[i] == i)

# ===== Example Usage =====
if __name__ == "__main__":
    dsu = DSU(7)

    dsu.unite(0, 1)
    dsu.unite(1, 2)
    dsu.unite(3, 4)
    dsu.unite(5, 6)

    print("0 and 2 connected?", dsu.same(0, 2))      # True
    print("0 and 3 connected?", dsu.same(0, 3))      # False
    print("Components:", dsu.count())                # 3

    dsu.unite(2, 3)
    print("After union(2,3):")
    print("0 and 3 connected?", dsu.same(0, 3))      # True
    print("Components:", dsu.count())                # 2
    print("Size of set containing 0:", dsu.component_size(0)) # 5

```

DSU with Rollback (Python)

```

class RollbackDSU:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.size = [1] * n
        self.history = [] # stack of (child, old_parent, root, old_size)

    def find(self, x: int) -> int:
        """Find WITHOUT path compression"""
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def unite(self, x: int, y: int) -> bool:
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            self.history.append((-1, -1, -1, -1)) # no-op marker
            return False

```

```

    if self.size[rx] < self.size[ry]:
        rx, ry = ry, rx

    self.history.append((ry, self.parent[ry], rx, self.size[rx]))
    self.parent[ry] = rx
    self.size[rx] += self.size[ry]
    return True

def rollback(self):
    child, old_parent, root, old_size = self.history.pop()
    if child != -1:
        self.parent[child] = old_parent
        self.size[root] = old_size

def snapshot(self) -> int:
    return len(self.history)

def rollback_to(self, snap: int):
    while len(self.history) > snap:
        self.rollback()

```

DSU on Grid (Python)

```

class GridDSU:
    def __init__(self, rows: int, cols: int):
        self.rows = rows
        self.cols = cols
        n = rows * cols
        self.parent = list(range(n))
        self.size = [1] * n
        self.components = n

    def _id(self, r: int, c: int) -> int:
        return r * self.cols + c

    def find(self, x: int) -> int:
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def unite(self, r1: int, c1: int, r2: int, c2: int) -> bool:
        x, y = self._id(r1, c1), self._id(r2, c2)
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False

        if self.size[rx] < self.size[ry]:
            rx, ry = ry, rx
        self.parent[ry] = rx
        self.size[rx] += self.size[ry]

```

```

        self.components -= 1
        return True

    def same(self, r1: int, c1: int, r2: int, c2: int) -> bool:
        return self.find(self._id(r1, c1)) == self.find(self._id(r2, c2))

    def get_components(self) -> int:
        return self.components

```

10. Code Explanation

DSU Class Breakdown

Constructor (DSU(int n)):

- Creates `parent` array of size `n` where `parent[i] = i` (each element is its own root).
- Creates `size` array of size `n` initialized to `1` (each set has size 1 initially).
- Time: $O(n)$.

find(int x):

- **Base case:** If `parent[x] == x`, then `x` is the root, return `x`.
- **Recursive case:** Recursively find the root of `parent[x]`, then set `parent[x]` to that root (path compression).
- **Why it works:** Path compression ensures that next time we call `find` on `x` or any node on the path, we go directly to the root in $O(1)$.
- **Edge case:** `x` is out of bounds — not handled here; caller ensures valid input.

unite(int x, int y):

- Find roots of both elements.
- If roots are same, return `false` (already connected, no change).
- Attach the root of the smaller set to the root of the larger set (union by size).
- Update the size of the new root.
- Return `true` (a union was performed).
- **Why union by size:** Without it, the tree could become a chain of length $O(n)$. With it, the height is $O(\log n)$.

same(int x, int y):

- Returns `find(x) == find(y)`. Simple and $O(\alpha(n))$.

count():

- Iterates through the `parent` array and counts elements `i` where `parent[i] == i` (roots).
- This gives the number of disjoint sets.
- **Alternative:** Maintain a `components` counter that decrements on each successful union.

DSU with Rollback Explanation

- **No path compression:** Path compression is destructive — it modifies the tree structure in a way that cannot be easily undone. Rollback DSU uses iterative find without compression.
- **History stack:** Each union saves the state before modification: (child, child's old parent, root, root's old size).
- **Rollback:** Pops the top of the stack and restores the old values. If the union was a no-op (already connected), the marker (-1, -1, -1, -1) is popped and ignored.
- **Snapshot:** Returns the current size of the history stack, which represents the number of unions performed. This allows rolling back to a specific point in time.

11. Complexity Analysis

Operation	Without Optimizations	With Path Compression	With Union by Size/Rank	With Both Optimizations
Find	$O(n)$	$O(\log n)$ amortized	$O(n)$	$O(\alpha(n))$ amortized
Union	$O(n)$	$O(\log n)$ amortized	$O(\log n)$	$O(\alpha(n))$ amortized
Construction	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$	$O(n)$	$O(n)$

What is $\alpha(n)$?

$\alpha(n)$ is the **inverse Ackermann function**. It grows so slowly that for all practical values of n (up to 10^{600}), $\alpha(n) \leq 5$. So we can treat it as **constant time** (effectively $O(1)$).

n	$\alpha(n)$
10^3	3
10^6	4
10^{10}	4
10^{100}	5

Detailed Breakdown

- **Find with path compression:** $O(\alpha(n))$ amortized per operation.
- **Union with both optimizations:** $O(\alpha(n))$ amortized — comprises two Finds + $O(1)$ work.
- **Space:** $O(n)$ for parent array + $O(n)$ for size/rank array = $O(n)$ total.
- **Best case:** All elements are already in one set — Find is $O(1)$ with path compression.
- **Worst case (without optimizations):** Union creates a chain of length n — Find is $O(n)$.
- **Worst case (with optimizations):** Practically $O(\alpha(n))$ — essentially constant.

Rollback DSU Complexity

Operation	Complexity
Find (no path compression)	$O(\log n)$

Operation	Complexity
Union	$O(\log n)$
Rollback	$O(1)$
Space	$O(n + \text{number of unions})$

12. Common Patterns

Pattern 1: Cycle Detection in Undirected Graph

How to identify: Problem asks "does adding this edge create a cycle?" or "is there a cycle in the graph?"

Approach: Iterate through edges. For each edge (u, v) , if $\text{find}(u) == \text{find}(v)$, a cycle is detected. Otherwise, $\text{union}(u, v)$.

Example problems: Redundant Connection, Graph Valid Tree, Detect Cycle in Undirected Graph.

Pattern 2: Number of Connected Components

How to identify: Problem asks "how many connected components are there?" or "count the number of groups."

Approach: Initialize DSU with n elements. For each connection, call unite . Track component count (decrement on each successful union, or count roots at the end).

Example problems: Number of Connected Components in an Undirected Graph, Friend Circles, Accounts Merge.

Pattern 3: DSU on Grid (2D Connectivity)

How to identify: Grid-based problem where adjacent cells (4-directional or 8-directional) can be merged based on some condition.

Approach: Map each cell (r, c) to a unique ID $r * \text{cols} + c$. Iterate through the grid and union adjacent cells that satisfy the condition.

Example problems: Number of Islands (DSU variant), Regions Cut by Slashes, Satisfiability of Equations.

Pattern 4: Kruskal's Minimum Spanning Tree (MST)

How to identify: Problem asks for "minimum cost to connect all nodes" or "minimum spanning tree."

Approach:

1. Sort all edges by weight.
2. Initialize DSU with n nodes.
3. Iterate through sorted edges. For each edge (u, v, w) , if $\text{find}(u) != \text{find}(v)$, add the edge to MST and $\text{union}(u, v)$.
4. Stop when MST has $n-1$ edges.

Example problems: Kruskal's MST, Connecting Cities with Minimum Cost, Min Cost to Connect All Points.

Pattern 5: Accounts Merge

How to identify: Problem where you have accounts with common information (emails, phone numbers) and need to merge accounts that share at least one common identifier.

Approach:

1. Map each email to the first account index that contains it.
2. For each subsequent account, for each email, if the email is already mapped, union the current account index with the mapped account index.
3. After processing, group emails by root and sort.

Example problems: Accounts Merge (LeetCode 721).

Pattern 6: Redundant Connection

How to identify: Graph starts as a tree, and one extra edge is added. Find the edge that can be removed to make it a tree again.

Approach: Iterate through edges. For each edge (u, v) , if $\text{find}(u) == \text{find}(v)$, this edge creates a cycle and is the redundant connection. Otherwise, $\text{union}(u, v)$.

Example problems: Redundant Connection (LeetCode 684), Redundant Connection II (LeetCode 685).

Pattern 7: DSU with Rollback (Offline Dynamic Connectivity)

How to identify: Problem asks about connectivity over time, where edges are added and removed, and queries are offline (you know all operations in advance).

Approach: Use divide-and-conquer on the time axis. Each edge is active over a time interval. Add the edge to the DSU when entering a segment, rollback when leaving. Answer queries at leaf nodes.

Example problems: Dynamic Graph Connectivity (Codeforces), Offline connectivity queries with deletions.

Pattern 8: DSU on Tree / Small-to-Large Merging

How to identify: Tree queries where you need to combine information from children to answer subtree queries. The "small to large" principle ensures $O(n \log n)$.

Approach:

1. For each node, maintain a data structure (set, map, etc.) with information about its subtree.
2. Process children. For each child, merge the smaller set into the larger set.
3. Answer queries for the current node after merging.

Example problems: Lomsat gelral (CF 600E), Tree Queries, Distinct Colors in a Subtree.

13. Common Mistakes

Mistake	Description	Fix
Forgetting path compression	Find is slow without it, leading to $O(n)$ per operation	Always add <code>parent[x] = find(parent[x])</code> or use iterative path halving
Forgetting union by size/rank	Tree becomes a chain of length n , worst-case $O(n)$	Always track size/rank and attach smaller to larger
Not using <code>find</code> in <code>unite</code>	Using <code>parent[x]</code> directly instead of <code>find(x)</code>	Always call <code>find(x)</code> to get the root
Swapping incorrectly	Swapping <code>rx</code> and <code>ry</code> after attaching the wrong way	Ensure: if <code>size[rx] < size[ry]</code> , swap before attaching
Not updating size	After union, forgetting to update <code>size[rx] += size[ry]</code>	Always update size after union
Off-by-one in grid mapping	Using <code>r * cols + c</code> with wrong dimensions	Double-check: <code>id = r * cols + c</code> , ensure <code>r < rows</code> , <code>c < cols</code>
Modifying parent during rollback find	Using path compression in rollback DSU	Rollback DSU must use iterative find without compression
Stack overflow with recursion	Deep recursion in <code>find</code> for large n	Use iterative find or increase stack size, or use path halving
Not resetting DSU for each test case	Parent array from previous test case leaks	Create a fresh DSU for each test case
Assuming 0-indexed vs 1-indexed	Input uses 1-indexed nodes but DSU is 0-indexed	Adjust: <code>unite(u-1, v-1)</code> if input is 1-indexed
Not handling duplicate edges	Processing the same edge twice	Either skip duplicates or check if already connected
Incorrect component count	Forgetting to decrement count on union	Track <code>components</code> variable, decrement on each successful union

14. Edge Cases

Edge Case	What Happens	How to Handle
Empty graph ($n = 0$)	DSU with 0 elements	Return 0 components, no operations
Single element ($n = 1$)	No unions possible	Find returns 0, <code>same(x,x)</code> is always true
All elements already connected	Every union returns false	DSU handles this naturally
Disconnected graph	Multiple roots remain	<code>count()</code> returns > 1

Edge Case	What Happens	How to Handle
Self-loop (edge from u to u)	<code>find(u) == find(u)</code> always	Skip or treat as cycle
Duplicate edges	Second union returns false	DSU naturally handles this
Large n (10^6 or more)	Recursion may overflow stack	Use iterative find or increase recursion limit
Negative values	Array index out of bounds	Map to non-negative indices or use hash map
Non-contiguous node labels	Labels like {1, 5, 100}	Use hash map (<code>unordered_map</code>) DSU
Grid with single row/column	Edge cases in grid neighbor checks	Check bounds: <code>r > 0, r < rows-1, c > 0, c < cols-1</code>
All unions are no-ops	Everything stays disconnected	Component count remains n
Maximum number of unions	n-1 unions connect everything	Component count becomes 1
Rollback on empty history	Undefined behavior	Check if history is non-empty before rollback

15. Variations

15.1 DSU with Path Halving (Iterative)

What changes: Instead of setting `parent[x] = find(parent[x])` recursively, we use a loop with path halving: `parent[x] = parent[parent[x]]`.

When used: When recursion depth is a concern (very large n) or when you prefer iterative code.

Importance: Useful for CP when n is large (10^7). Avoids stack overflow.

```
int find(int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]]; // path halving
        x = parent[x];
    }
    return x;
}
```

15.2 DSU with HashMap (Disjoint Sets for Non-Integer Elements)

What changes: Instead of an array, use `unordered_map<int, int>` for parent and size. Elements can be any hashable type.

When used: When nodes are labeled with non-integer IDs (strings, large numbers, scattered indices).

Importance: Common in problems where labels are not contiguous.

```
class HashMapDSU {
    unordered_map<int, int> parent, sz;
public:
    int find(int x) {
        if (!parent.count(x)) parent[x] = x, sz[x] = 1;
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    bool unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;
        if (sz[rx] < sz[ry]) swap(rx, ry);
        parent[ry] = rx;
        sz[rx] += sz[ry];
        return true;
    }
};
```

15.3 Weighted DSU (DSU with Distance to Root)

What changes: Each node stores the distance/weight to its parent. `find` returns the root and also updates the weight.

When used: Problems where you need to know the relative difference between elements in the same set (e.g., "is the sum of weights equal?", "relative ordering").

Importance: Used in problems like "Possible Friends" or "Equations with Relative Values."

```
class WeightedDSU {
    vector<int> parent, diff; // diff[x] = weight from x to parent[x]
public:
    WeightedDSU(int n) : parent(n), diff(n, 0) {
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x) {
            int p = find(parent[x]);
            diff[x] += diff[parent[x]];
            parent[x] = p;
        }
        return parent[x];
    }

    // weight(x) - weight(y) = w
    bool unite(int x, int y, int w) {
```

```

    int rx = find(x), ry = find(y);
    if (rx == ry) return (diff[x] - diff[y]) == w;
    parent[rx] = ry;
    diff[rx] = diff[y] + w - diff[x];
    return true;
}
};

```

15.4 Persistent DSU

What changes: Maintains version history of the DSU using persistent data structures or copy-on-write.

When used: When you need to query connectivity at any point in time (not just the latest state).

Importance: Rare in CP, but appears in some advanced problems.

15.5 DSU with Delete (Using Lazy Deletion)

What changes: Instead of actually deleting, mark nodes as "deleted" and adjust the DSU. This is complex and not standard.

When used: When you need to support deletion of elements.

Importance: Rare. Usually solved with rollback or offline techniques instead.

15.6 DSU on Tree (Sack / Small-to-Large)

What changes: This is actually a tree DP technique, not a DSU variant. It uses the "small to large" merge principle but on tree data structures (sets, maps, etc.).

When used: Subtree queries where you need to combine information from children.

Importance: Very important for CP (Codeforces rating 1800-2200).

16. Related Algorithms/Data Structures

Algorithm/DS	Connection to DSU	When to Choose
DFS/BFS	Both can find connected components	DSU is better for dynamic connectivity (edges added over time). DFS/BFS is better for static graphs and when you need the actual traversal order.
Kruskal's MST	Uses DSU internally	DSU is the core of Kruskal's. Use Kruskal when edges are given and you need MST.
Prim's MST	Alternative to Kruskal's	Use Prim for dense graphs ($O(E \log V)$ vs $O(E \log E)$). Kruskal + DSU is better for sparse graphs.
Tarjan's SCC	Both find connectivity	Tarjan finds strongly connected components (directed graphs). DSU is for undirected connectivity.

Algorithm/DS	Connection to DSU	When to Choose
Segment Tree	No direct relation	Segment Tree handles range queries and updates. DSU handles set connectivity.
Fenwick Tree (BIT)	No direct relation	BIT handles prefix sum queries. DSU handles connectivity.
Link-Cut Tree	Supports dynamic connectivity with edge deletions	Link-Cut Tree is more powerful but much more complex. Use DSU for simplicity when deletions aren't needed.
Euler Tour Tree	Supports dynamic connectivity with edge deletions	Similar to Link-Cut Tree. Overkill for most problems. DSU is simpler and faster for insert-only.
BFS for Connected Components	Same purpose, different approach	DSU is better for online queries (edges added over time). BFS is better for offline queries on a static graph.
Topological Sort	Both deal with graphs	Topological sort is for directed acyclic graphs. DSU is for undirected connectivity.

How to Choose

- **Need to add edges dynamically and check connectivity?** → DSU
- **Need to remove edges too?** → Link-Cut Tree or offline DSU with rollback
- **Need directed graph connectivity?** → Tarjan's SCC (not DSU)
- **Need paths between nodes?** → BFS/DFS or Dijkstra (not DSU)
- **Need MST?** → Kruskal's (uses DSU) or Prim's
- **Need range queries on arrays?** → Segment Tree / Fenwick (not DSU)
- **Need subtree queries on tree?** → DSU on Tree (Sack) or Euler tour + Segment Tree

17. Practice Problems

Easy

Problem	Platform	Main Idea	Difficulty
Number of Connected Components in an Undirected Graph	LeetCode 323	Basic DSU: count components after processing all edges	Easy
Find if Path Exists in Graph	LeetCode 1971	Use DSU to check if two nodes are connected	Easy

Medium

Problem	Platform	Main Idea	Difficulty
Redundant Connection	LeetCode 684	Cycle detection with DSU; find the edge that creates a cycle	Medium

Problem	Platform	Main Idea	Difficulty
Accounts Merge	LeetCode 721	DSU on emails; map emails to account indices; union on shared emails	Medium
Number of Islands (DSU Approach)	LeetCode 200	Grid DSU: union adjacent land cells; count components	Medium

Hard

Problem	Platform	Main Idea	Difficulty
Redundant Connection II	LeetCode 685	Directed graph variant; DSU + case analysis (node with two parents vs cycle)	Hard
Making A Large Island	LeetCode 827	Grid DSU + component size tracking; try flipping each 0 to 1	Hard
Dynamic Graph Connectivity	Codeforces (CF 1217F)	DSU with rollback + offline divide-and-conquer; forced online variant	Hard

Additional Practice

Problem	Platform	Main Idea	Difficulty
Graph Valid Tree	LeetCode 261	DSU to check if graph is a tree (connected + no cycles)	Medium
Friend Circles	LeetCode 547	Count components in a friendship matrix	Medium
Regions Cut By Slashes	LeetCode 959	Grid DSU with 4 sub-cells per cell	Medium
Satisfiability of Equality Equations	LeetCode 990	DSU for equalities, then check inequalities	Medium
Min Cost to Connect All Points	LeetCode 1584	Kruskal's MST with DSU	Medium
Connecting Cities With Minimum Cost	LeetCode 1135	Kruskal's MST with DSU	Medium
Lomsat gelral	Codeforces 600E	DSU on tree (small-to-large merging)	Hard
Number of Good Paths	LeetCode 2421	DSU with sorting by value	Hard
Minimum Spanning Tree	CSES	Kruskal's MST with DSU	Medium
Road Reparation	CSES	Kruskal's MST with DSU	Medium

18. Interview Explanation

"Disjoint Set Union, also called Union-Find, is a data structure that tracks a collection of elements partitioned into disjoint sets. It supports two main operations.

Find returns the representative or root of the set containing a given element. With path compression, we make every node on the path point directly to the root, which flattens the tree.

Union merges two sets. We find the roots of both elements, and if they are different, we attach the root of the smaller set under the root of the larger set — this is union by size. This ensures the tree stays shallow.

With both optimizations, each operation runs in amortized $O(\alpha(n))$ time, which is essentially constant. The space complexity is $O(n)$.

I use DSU whenever I need to solve connectivity problems dynamically — like detecting cycles in an undirected graph, finding connected components, building a minimum spanning tree with Kruskal's algorithm, or problems like Accounts Merge and Redundant Connection.

One important thing to remember: DSU works for undirected graphs only. For directed graphs, you need different algorithms like Tarjan's for strongly connected components."

19. Revision Notes

Key Idea

- DSU maintains disjoint sets using a tree structure.
- Each set has a **root** (representative). `parent[i] = i` means `i` is a root.
- Two optimizations: **path compression** (flatten trees) + **union by size/rank** (attach smaller to larger).

Core Operations

- **Find(x)**: Return root of `x`. `if (parent[x] != x) parent[x] = find(parent[x]); return parent[x];`
- **Union(x, y)**: Find roots. If different, attach smaller to larger. Update size.
- **same(x, y)**: `find(x) == find(y)`

Template Reminder

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) { p.resize(n); sz.resize(n, 1); iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        p[y] = x; sz[x] += sz[y]; return true;
    }
};
```

Complexity

- **With both optimizations:** $O(\alpha(n))$ per operation (essentially $O(1)$).
- **Space:** $O(n)$.

Common Traps

- ❌ Always call `find(x)`, not `parent[x]`, in `unite`.
- ❌ Always update size after union.
- ❌ Rollback DSU: no path compression.
- ❌ Grid DSU: `id = r * cols + c`.
- ❌ 1-indexed input: `unite(u-1, v-1)`.

When to Use

- Dynamic connectivity ✓
- Cycle detection ✓
- Kruskal's MST ✓
- Number of components ✓
- Accounts merge ✓

When NOT to Use

- Directed graphs ✗
- Edge deletions (without rollback) ✗
- Path queries ✗
- Distance queries ✗

20. Final Cheat Sheet

DSU — Disjoint Set Union / Union-Find

DSU CHEAT SHEET
WHEN TO USE • Dynamic connectivity (add edges, query connectivity) • Cycle detection in undirected graphs • Kruskal's Minimum Spanning Tree • Number of connected components • Equivalence relations (transitive closure) • Grid connectivity (2D → 1D mapping)
MAIN OPERATIONS • <code>find(x)</code> → root of <code>x</code> (path compression) • <code>unite(x,y)</code> → merge sets (returns true if merged) • <code>same(x,y)</code> → <code>find(x) == find(y)</code> • <code>count()</code> → number of roots / components

COMPLEXITY

Operation	Complexity
Find (optimized)	$O(\alpha(n)) \sim O(1)$ amortized
Union (optimized)	$O(\alpha(n)) \sim O(1)$ amortized
Construction	$O(n)$
Space	$O(n)$

KEY CODE (C++17)

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    }
    bool unite(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (sz[x] < sz[y]) swap(x, y);
        p[y] = x; sz[x] += sz[y]; return true;
    }
};
```

KEY CODE (Python)

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n))
        self.sz = [1] * n
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]] # path halving
            x = self.p[x]
        return x
    def unite(self, x, y):
        x, y = self.find(x), self.find(y)
        if x == y: return False
        if self.sz[x] < self.sz[y]: x, y = y, x
        self.p[y] = x; self.sz[x] += self.sz[y]; return True
```

IMPORTANT EDGE CASES

- Self-loop: `find(u) == find(u)` → always cycle
- Duplicate edges: union returns false
- 1-indexed input: `unite(u-1, v-1)`
- Non-contiguous labels: use `unordered_map`
- Grid: `id = r * cols + c`
- Rollback: no path compression, use iterative find
- Large n: use iterative find to avoid stack overflow

COMMON PATTERNS

- Cycle detection → if `find(u) == find(v)`, cycle exists
- Component count → decrement on each successful union
- Kruskal's MST → sort edges, unite if different roots

- Accounts merge → map email to first account index, union
- Redundant connection → first edge where $\text{find}(u) == \text{find}(v)$
- Grid DSU → map (r, c) to $r * \text{cols} + c$, union adjacent cells

COMMON MISTAKES

- ✗ Forgetting path compression or union by size
- ✗ Using `parent[x]` instead of `find(x)` in unite
- ✗ Not updating size after union
- ✗ Using path compression in rollback DSU
- ✗ Off-by-one in grid mapping

Pro Tip: DSU is one of the most elegant data structures in competitive programming. The combination of path compression and union by size makes it essentially constant time, yet the implementation is under 15 lines. Master this — it appears in at least one problem in almost every coding contest and many SDE interviews.